Firmware Lab 3
Acquisition and display of ambient temperature

May 19th, 2026

WARNING!!!

INSTRUCTIONS MUST BE CAREFULLY READ BEFORE PERFORMING ANY OPERATION

Lab goals

The current lab has the goal to learn how to perform periodic measurements of ambient temperature, and to show the value on the LCD screen of the VirtLAB board.
This task is accomplished through the following materials:

- A project template for the user MCU, in which the firmware is partially pre-configured. Please notice that the firmware must be changed by users, as it is missing C code sections needed to perform the required task.
- A VirtLAB board, on which previous firmware is loaded and run.

- A Digital Storage Oscilloscope (DSO), used to perform real word measurements on the VirtLAB board.
- An NTC resistor, and three 10kΩ fixed resistors.
- A breadboard board, to connect electronic components to the VirtLAB.

To correctly accomplish the required goals, a full reading of this documents is mandatory **before** starting to operate on the PC.

Hardware configuration

Temperature measurements are obtained reading the resistance variation of a NTC resistor. This component has a value which varies with the temperature. It has a 10kΩ resistance at 25°C. The value decreases by 4% for each increase of 1°C of the component temperature.

As the VirtLAB board is not able to perform direct resistance measurements, this value must be converted to a voltage, through a simple resistive divider, realized using the NTC as the lower side, and a fixed 10kΩ resistor as the upper side. The high side of the divider is connected to the power supply of the microcontroller, while the lower side is connected to ground. The output voltage of the divider will then be:

$$V_{NTC} = V_{cc}\frac{R_{NTC}}{R_{NTC} + R_{10k}} \tag{1}$$

where V_{cc} is the power supply voltage, nominally 3.3V on the VirtLAB board.

To take in account possible tolerances in power supply, this voltage must be measured, too, and used to adjust the sensor value. Again, a fixed resistor divider, with fixed ratio, is used, using two 10kΩ resistors. In this case the output voltage of the divider will be:

$$V_{power} = V_{cc}\frac{R_{10k}}{R_{10k} + R_{10k}} = \frac{V_{cc}}{2} \tag{2}$$

Using the last equation, the unknown value of V_{cc} can be eliminated, and substituted by the known, and measured, V_{power}.

The outputs are connected to the inputs of two different ADC inside the microcontroller. V_{NTC} is connected to input 3 of ADC1. V_{power} is connected

to input 4 of ADC2. The two ADCs return a value on 12 bit, which is a binary representation of the following quantity:

$$ADC_{result} = (2^{12} - 1)\frac{V_{in}}{V_{ref}} \tag{3}$$

where V_{in} is the value of the voltage at the input of the ADC, and V_{ref} is a fixed value, which is nominally equal to 2.048V.

The two resistive dividers, NTC and fixed one, must be assembled on a breadboard external to the VirtLAB board, and connected through the provided plug/socket cables to the analog port, located on J801. On this port, analog inputs, 3V3 and GND connections are present.

The resulting schematic is shown in figure 1.

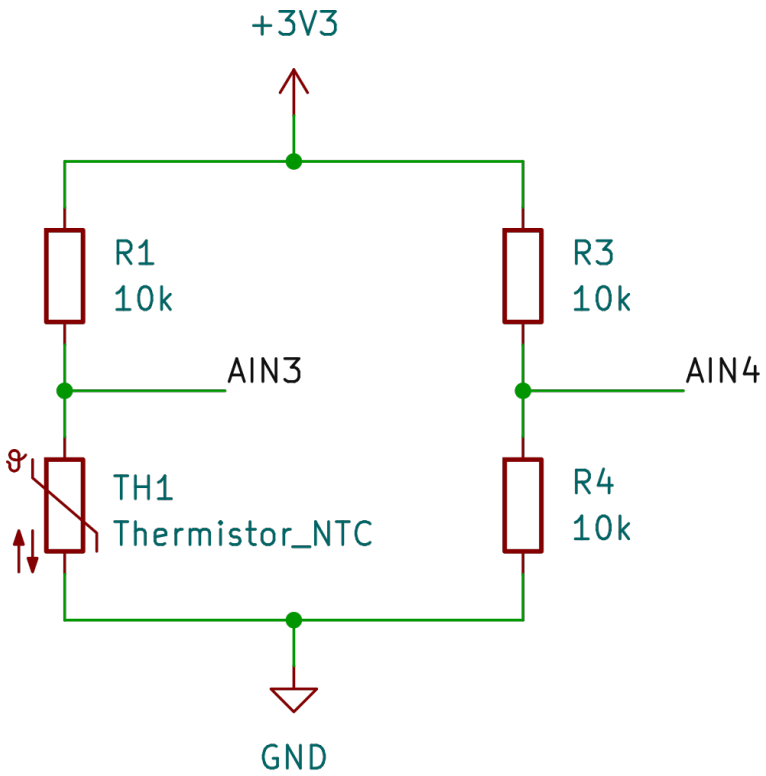

Figure 1: Schematic of dividers and connections to the VirtLAB board, connector J801. AIN3 and AIN4 are the inputs to the ADC converters

Firmware structure

The firmware uses the capabilities of FreeRTOS to split overall computations in three different tasks, which communicate one each other through queues. To simplify student work, the following tasks are already defined in the firmware template:

- Acquisition task (acquisition.c). This module must periodically acquire, at a 10 Hz rate, the values at the input of ADCs, and compute the NTC resistor temperature. Data representation must be sufficient to include at least one decimal digit. Result unit of measure must be °K. Periodic acquisition must be performed using the interrupt of TIM1 to start acquisitions.
- Display task (display.c). This module receives a value of temperature, in SI units, and display it on the LCD screen of the VirtLAB.
- Default task (application.c). This task manage data exchange among the previous described tasks, according to the flow described below.

Data exchange between tasks is managed by the following queues, already instantiated in the template:

- AcquisitionQueue. This queue is written by the acquisition task whenever a new temperature value is ready. It is read by the default task.
- DisplayQueue. This queue is written by the default task. Whenever a temperature value is read from the acquisition queue, it is duplicated and written to the display queue.

The three tasks, even if already instantiated, are empty, and the student must fill them with C source code, suitable to accomplish the required functionalities.

ADC conversion must be driven through the HAL_ADC_xxx API. ADC converters are already pre-configured by the given template, and only the functions to start conversion and to get results must be inserted in the source code.

LCD visualization can be realized through the usage of convenience functions, included in io.c/io.h, allowing to write numbers on the LCD screen of the VirtLAB.

Queue write and read functions are documented in cmsis_os2.c and cmsis_os2.h, where they are **osMessageQueuePut()** for writing, and **osMessageQueueGet()** for reading. Please notice that they are already protected by semaphores, and no additional care is needed for multi thread programming.

Last, it is strongly discouraged the usage of waiting loop inside C functions, as well as the direct invocation of HAL API function from inside an interrupt handler. As a possible solution, event programming must be used, as explained during the lessons.

The related API, described in cmsis_os2.c and cmsis_os2.h, is mainly based on the two functions **osEventFlagsWait()**, used to stop a task until an event is received, and **osEventFlagsSet()**, used to raise an event flag. The latter can be called from an interrupt handler, too.

Lab report content

The user must provide a detailed description of the source code structure, describing operations executed by every task, including a flowchart for each process. The commented C source code must be attached, too.